

Closed draft, 2026-08-26. Single-file merge of the per-section drafts (paper_A*_draft_2026-08-25.md) in submission order, updated after the E230 gate was **wired into the production checker, built from source, and verified** (Madaros v0.80.0; integration commit 4ac63da51f on base 06e85a6ada, branch fable/ns-antigarbling-integration-20260825, xai+zai math-reviewed). §8’s evaluation now reports measured wired-compiler results; only interprocedural parameter projection (§10) remains marked as such. **Update 2026-08-31:** RQ4’s two-compartment flip rate is measured (§8.4; 34.2 % of true WARNs silenced in the $\rho = 1$ interval sum, 0 % in the phase decomposition where the covariance is negative). **Update 2026-08-30:** the NS-extended metatheory of §6 is now fully mechanized (formal/lean4/EpistemicEffectsNS.lean) — Lemma 2, NS progress/preservation, exactness preservation and Theorem 6.4 itself carry kernel proofs; §6.4’s table and §6.3 are updated accordingly. Target: PLDI/OOPSLA. Grounding index and prior-art citations: paper_A_README.md; prior-art sign-off: paper_A_priorart_gate_signoff_2026-08-25.md. Notation is unified: $\mathbb{m}$ =mean, $\mathbb{v}$ =variance, $\text{Cov}/\langle \cdot, \cdot \rangle$ =covariance/inner product, $\text{Knowledge}\langle T, N \rangle$ =epistemic type with noise-set N .

Manufacturing Precision Is a Type Error

Compile-Time Anti-Garbling for Uncertainty-Typed Languages

Anonymous Author(s)

Abstract

Libraries that propagate measurement uncertainty — `Measurements.jl`, `Uncertain(T)`, GUM implementations — assume the operands of every arithmetic operation are independent. When they are not, the propagated uncertainty is silently *understated*: the program fabricates precision it has not earned. We show this is not a corner case but a shipping defect. In one production uncertainty library, `mul(x, x)` returns $2x^2 \cdot \text{var}$ while `square(x)` returns the correct $4x^2 \cdot \text{var}$ — the same mathematical operation, two formulas, with nothing routing $x \cdot x$ to the sound one; the correlated add/sub pair exhibits a matching directional asymmetry (add understates, sub stays conservative). We recast the problem through the Blackwell / data-processing refinement order already standard in quantitative information flow: a sound uncertainty operation is a *garbling* (information-losing); understating variance is an *anti-garbling* (information-creating), which no correct program may do. We give a type system for an uncertainty-typed language that carries the **noise-symbol source-set** of each value in its type — reusing the source-identity idea of affine arithmetic, but in the type rather than in an external analyzer — and turns the independence assumption into a **checked precondition**: an independence-assuming operator over operands with non-disjoint (or unknown) source-sets is rejected unless a proved-disjoint certificate holds. We mechanize the soundness theorem in Lean for the core calculus (kernel-checked, Mathlib-free): the naive scalar operation is sound iff the operand covariance is zero, the tracked source-set over-approximates the true support, and along every evaluation of a well-typed program each uncertainty value reports its true first-order variance. We implement the discipline in the Sounio compiler and show it eliminates the defect class while accepting correlated-aware code, evaluated on a physiologically-based pharmacokinetic model where every inter-compartmental sum shares measured rate constants.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference'17, July 2017, Washington, DC, USA

© 2026 Association for Computing Machinery.

ACM ISBN 978-x-xxxx-xxxx-x/YY/MM...\$15.00

<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

ACM Reference Format:

Anonymous Author(s). 2026. Manufacturing Precision Is a Type Error: Compile-Time Anti-Garbling for Uncertainty-Typed Languages. In *Proceedings of ACM Conference (Conference'17)*. ACM, New York, NY, USA, 17 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Introduction

A measurement is never a number; it is a number and a doubt. Scientific and engineering software that takes measurements seriously has, over the last decade, learned to keep the doubt attached: *uncertainty-typed* languages and libraries — `Uncertain(T)`, `Measurements.jl`, GUM tooling — give a quantity a type like `Knowledge{f64}` that carries its variance, and propagate that variance automatically through arithmetic, so that a result arrives already annotated with how much it can be trusted. The promise is that uncertainty becomes a first-class, compiler-tracked property rather than a comment in a lab notebook.

That promise rests on a hypothesis the type never states and the compiler never checks: that the operands of every operation are **statistically independent**. The propagation laws these systems implement — $\text{Var}(X+Y) = \text{Var}(X) + \text{Var}(Y)$, $\text{Var}(XY) \approx Y^2\text{Var}(X) + X^2\text{Var}(Y)$ — are the independence special cases of the general laws, which carry an additional covariance term $\pm 2\text{Cov}(X, Y)$. When the operands are correlated, the covariance term is real and the shipped formula omits it. For addition and multiplication the omission runs in one direction: the reported variance is *smaller* than the truth. The program claims more precision than its inputs justify.

This failure is uniquely resistant to discovery, for a reason that has nothing to do with how carefully the code is written. An understated variance produces a **tighter error bar** — a more confident, more precise-looking answer. It is the one kind of wrong result that never looks wrong. A test asserting the value is correct passes; a test asserting the uncertainty is not too large passes; only a test written by someone who already knows the correct, larger variance can fail — that is, only someone who has already found the bug. The defect hides in exactly the blind spot testing cannot cover, because the symptom is indistinguishable from success.

It is not hypothetical. In a production uncertainty library we examined, the operation $x \cdot x$ has two implementations that disagree. Multiplying a value by itself through the general product operator returns a variance of $2m^2v$; squaring the same value through the dedicated square operator returns $4m^2v$ — twice as much, and correct. The two are the same mathematical operation; both ship; nothing in

the library routes $x \cdot x$ to the sound one. The discrepancy is precisely the covariance term the product operator drops, which for $x \cdot x$ is not negligible but equal to the entire result. A programmer who writes $x * x$ gets, silently, half the uncertainty they are owed. The same omission makes correlated addition understate and — revealingly — makes correlated *subtraction* overstate, an asymmetry we show is the fingerprint of the underlying error rather than random sloppiness.

Our thesis is that manufacturing precision is a type error, and it is the same error as manufacturing information. Quantitative information flow has a mature theory of when a computation is allowed to change what is known: the Blackwell / data-processing order, in which an admissible transformation is a *garbling* — it may lose information but never create it. Understating variance is the forbidden move, an **anti-garbling**: it reports less uncertainty, hence more information, than the operands contain. Read this way, the library's $x + x$ is not merely inaccurate; it violates a conservation law. And conservation laws are what type systems are good at enforcing.

We enforce it. The reason the library cannot condition on $\text{Cov}(X, Y)$ is that its scalar representation (mean, variance) cannot express which measurements a value depends on. We restore that information to the type. Borrowing the *noise-symbol* representation of affine arithmetic — where a value is an affine form over independent per-source symbols and two values are correlated exactly when they share a symbol — we carry each value's **source-set** in its type, $\text{Knowledge}(T, N)$. An independence-assuming operator is then well-typed only when its operands' source-sets are provably disjoint; otherwise it is a compile-time error. The programmer discharges it either with a proof that the operands are uncorrelated or by switching to an operator that takes the correlation explicitly. The unsound path does not type-check.

What we claim, and what we do not. Neither half of the mechanism is new on its own, and we are explicit about this up front so the contribution is not mistaken for either piece. Tracking noise-symbol identity to preserve correlations is the defining idea of affine arithmetic (Comba & Stolfi 1993) and of the Fluctuat static analyzer (Goubault & Putot) — but there it lives in an external analyzer that *computes an enclosure* and rejects nothing. The Blackwell/garbling soundness order is standard in quantitative information flow (McIver, Morgan & Smith; Alvim et al.) — but there it orders confidentiality channels and measures leakage. Our contribution is their **intersection**, which neither field occupies: the source-set lifted *into the type* of an uncertainty-typed language, and the independence assumption of uncertainty arithmetic made a *checked precondition* whose violation is a type error with a discharging certificate. We track source identity like affine arithmetic, forbid anti-garbling like QIF, and enforce it in the type like neither.

Contributions.

1. **A characterization of the defect class** — independence assumed and unchecked in uncertainty propagation — grounded in a shipping library, together with its framing as a Blackwell anti-garbling, including the add/sub directional signature that the framing predicts and the source confirms (§2, §4).
2. **A type discipline** that carries the noise-symbol source-set in the type and makes the independence assumption a checked precondition: the E230 rejection rule, the noise-set lattice and its transfer functions, and the proved-disjoint / correlation-aware escape valves (§5).
3. **A soundness result** — no well-typed program contains a first-order anti-garbling — mechanized in Lean for the NS-extended *core calculus* — the covariance-exactness criterion in general form, the soundness of the source-set abstraction, NS type safety, and the theorem itself (along every evaluation of a well-typed program each Knowledge value reports its true first-order variance) — with the correspondence between that calculus and the production checker's E230 rule established by test and sabotage gates, not by proof (§6.4).
4. **An implementation and evaluation** in the self-hosted Sounio compiler: running prototypes and a kernel-checked model that reproduce the defect, establish that the rejection is *caused* by source-set propagation (a sabotage-controlled experiment), and a clinical case study — vancomycin therapeutic-drug-monitoring — where understatement silences the safety warning the system exists to raise (§7, §8).

The guarantee is deliberately narrow — first-order, conservative, in the variance channel — and §10 states its boundaries plainly. Within that scope it eliminates exactly the defect class of §2: the answer that looks too precise to be true becomes the one the compiler refuses to print.

2 The defect class, by example

The uncertainty library we study is not careless. Every arithmetic operation in `knowledge.sio` is annotated with the exact GUM propagation formula it implements, and each formula is *correct* — under one assumption stated nowhere in the type and checked nowhere in the code: that the operands are **statistically independent**. The library ships the independence special case of each GUM formula as if it were the general law. Where the assumption fails, the propagated variance is not merely imprecise; it is biased in a specific, dangerous direction — *downward*. The program reports more precision than its inputs justify.

We make this concrete with four operations from a single file, then show why the bias survives testing, and close with the realistic case that motivates the whole paper.

2.1 The same operation, two variances

Consider squaring an epistemic value. The library offers two ways to compute $x \cdot x$:

```
// GUM delta method: Var(XY) = Y^2*Var(X) + X^2*Var(Y)
knowledge.sio:110
pub fn ep_mul(a: &Epistemic, b: &Epistemic) -> Epistemic {
  let new_var = b.val * b.val * a.variance + a.val * a.val
  * b.variance // :112
  ...
}
```

```
// Var(X^2) = 4X^2*Var(X)
knowledge.sio:150
pub fn ep_square(a: &Epistemic) -> Epistemic {
  Epistemic {
    val: a.val * a.val,
    variance: 4.0 * a.val * a.val * a.variance,
    // :154
    ...
  }
}
```

Evaluate both on the same $x = (m, v, \cdot)$:

Expression	Formula applied	Variance
ep_mul(&x, &x)	$b^2 \cdot \text{Var}(a) + a^2 \cdot \text{Var}(b)$ with $a = b = x$	$m^2v + m^2v = 2m^2v$
ep_square(&x)	$4 \cdot a^2 \cdot \text{Var}(a)$	$4m^2v$

ep_square is right: for $Y = X^2$, the delta method gives $\text{Var}(Y) = (dY/dX)^2 \cdot \text{Var}(X) = (2X)^2 \cdot \text{Var}(X) = 4m^2v$. ep_mul(&x, &x) returns **half** of that. The discrepancy is not rounding — it is the entire covariance term. The general delta-method formula for a product is

$$\text{Var}(XY) = Y^2 \cdot \text{Var}(X) + X^2 \cdot \text{Var}(Y) + 2XY \cdot \text{Cov}(X, Y).$$

ep_mul drops the last term. When X and Y are independent, $\text{Cov} = 0$ and the drop is exact. When Y is X , $\text{Cov}(X, X) = \text{Var}(X) = v$, the missing term is $2 \cdot m \cdot m \cdot v = 2m^2v$, and the two formulas differ by exactly that amount: $4m^2v - 2m^2v = 2m^2v$. The library computes both, exposes both, and **nothing routes $x \cdot x$ to the sound one**: ep_mul(&x, &x) is legal, well-typed, and understates the variance of a squared quantity by a factor of two.

2.2 The asymmetry the frame predicts

The same omission runs through addition and subtraction, and it produces a *directional* signature that is itself evidence the diagnosis is right.

```
// GUM: Var(X+Y) = Var(X) + Var(Y)
knowledge.sio (ep_add)
pub fn ep_add(a: &Epistemic, b: &Epistemic) -> Epistemic {
  ... variance: a.variance + b.variance, ...
  // :96
}
```

```
// GUM: Var(X-Y) = Var(X) + Var(Y)
knowledge.sio:101
pub fn ep_sub(a: &Epistemic, b: &Epistemic) -> Epistemic {
  ... variance: a.variance + b.variance, ...
  // :105
}
```

Both add the operand variances. The general laws are $\text{Var}(X \pm Y) = \text{Var}(X) + \text{Var}(Y) \pm 2 \cdot \text{Cov}(X, Y)$: addition *adds* twice the covariance, subtraction *subtracts* it. Evaluate each on maximally-correlated operands ($Y = X$, $\text{Cov} = v$):

Expression	Library	True ($\rho = 1$)	Error	Direction
ep_add(&x, &x) = $2x$	2v	$\text{Var}(2X) = 4v$	-2v	understates
ep_sub(&x, &x) = 0	2v	$\text{Var}(0) = 0$	+2v	overstates
ep_mul(&x, &x) = x^2	$2m^2v$	$4m^2v$	$-2m^2v$	understates
ep_square(&x) = x^2	$4m^2v$	$4m^2v$	0	sound

The asymmetry is the tell. Correlated addition and multiplication **understate** — they manufacture precision, the failure that matters. Correlated subtraction **overstates** — it is wrong, but wrong in the safe direction, reporting *more* uncertainty than the truth. A library that were simply buggy would err in both directions at random. This one errs in exactly the direction predicted by the sign of the dropped covariance term: +2Cov for add, -2Cov for sub. §4 names this the anti-garbling signature — an operation may only *lose* information (overstate uncertainty), never *create* it (understate) — and the add/sub split is that criterion read off the source.

A note on the *99/100, *98/100 confidence multipliers (:97, :116, ...): these decay a separate integer confidence field and are orthogonal to the variance defect. They are also heuristic rather than derived (a limitation we return to in §10), but they neither cause nor mask the understatement analyzed here.

2.3 Why tests do not catch it

An understated variance is invisible to ordinary testing for a structural reason: **the wrong answer is the more attractive one**. A tighter error bar reads as a better result. Nothing about ep_add(&x, &x) returning 2v instead of 4v looks like a failure — the value $2x$ is exact, the confidence field is populated, the number is simply *more precise*. A regression test asserting “the answer is within tolerance” passes; a test asserting “the uncertainty is not too large” passes; only a test that already knows the *correct* larger variance —

i.e. a test written by someone who has already spotted the bug — fails. The library’s own test suite (knowledge.sio:290–310) checks `ep_add`, `ep_sub`, and `ep_mul` on **independent** operands, where every formula is exact, and so certifies nothing about the correlated case.

The failure is therefore not a coding slip that better testing would have caught. It is a **missing precondition**: the operations are sound on a domain (independent operands) that the type system neither records nor enforces, and are silently applied outside it. This is precisely the shape a type discipline can address and testing cannot — the defect is a property of *which values may flow into which operation*, not of any single execution.

2.4 The instance that matters: shared provenance in a PBPK model

The correlated case is not exotic. It is the *normal* case whenever two quantities are computed from a common measurement — which, in a physiologically-based pharmacokinetic (PBPK) model, is every pair of compartment states, because they all descend from the same measured clearance and volume parameters.

Consider a two-compartment model reporting total drug exposure as the sum of central and peripheral AUC:

```
auc_total = ep_add(&auc_central, &auc_peripheral)
```

Both `auc_central` and `auc_peripheral` are derived, through the model’s ODE solution, from the *same* measured elimination rate constant k . Their errors are strongly positively correlated: an overestimate of k pushes both AUCs the same way. The true variance of the sum includes $+2 \cdot \text{Cov}(\text{auc_central}, \text{auc_peripheral}) > 0$; `ep_add` discards it. Every inter-compartmental sum in the model therefore reports a variance smaller than the truth, and the understatement compounds with each additional compartment.

The clinical consequence is the opposite of academic. The dissertation’s therapeutic-drug -monitoring result (vancomycin, rapamycin) issues a WARN when the *upper* credible bound on exposure crosses into toxicity even though the point estimate reads safe. A variance that is understated at every summation **shrinks that credible bound**, and the WARN that should fire — the entire safety value of propagating uncertainty — does not. The defect does not just weaken a number in a table; it silences the alarm the system exists to raise. §8 quantifies this on the real model; here it fixes the stakes: manufacturing precision in an uncertainty library is not a cosmetic inaccuracy but a safety failure, and the operation that commits it type-checks today.

2.5 Verbatim-source appendix (for the artifact / referee)

Numeric verification (for $x = (m, v, \cdot)$): `ep_add(&x, &x)`
`.variance=v+v=2v` vs `Var(2X)=4v`; `ep_mul(&x, &x)` .variance=

Item	Location	Content	
ep_add variance	stdlib/epistemic/ knowledge.sio:96	a.variance+b. variance	496
ep_sub variance	:105	a.variance+b. variance	497
ep_mul variance	:112	b.val*b.val*a. variance+a.val*a. val*b.variance	498
ep_square variance	:154	4.0*a.val*a. val*a.variance	499
GUM comments (in- tent)	:101, :110, :150	correct formulas, in- dependence implicit	500
tests use indepen- dent operands only	:290--310	correlated case untested	501

$m^2v+m^2v=2m^2v$ vs `ep_square(&x).variance=4m^2v`; under-
statement gaps $2v$ and $2m^2v$ equal $2 \cdot \text{Cov}(X, X)$ and $2 \cdot m^2 \cdot \text{Cov}(X, X)$
respectively — the exact dropped covariance terms.

3 Preliminaries

We fix four notions the rest of the paper uses. None is new here; §9 places them in the literature.

Uncertainty types (the host). We work in a language where a quantity carries its uncertainty in the type: an *epistemic value* has type $\text{Knowledge}\langle T \rangle$ and, at runtime, a payload of type T together with metadata — for our purposes a *variance* $v = \text{Var}(X) \geq 0$ (and a confidence field, orthogonal to soundness, which we ignore). Arithmetic on $\text{Knowledge}\langle T \rangle$ propagates the variance automatically. This is the setting of `Uncertain`, `Measurements.jl`, and GUM tooling (§9.3).
GUM propagation and its hidden hypothesis. The propagation laws these systems implement are the ISO GUM first-order (delta-method) rules: $\text{Var}(X+Y) = \text{Var}(X) + \text{Var}(Y) + 2\text{Cov}(X, Y)$, $\text{Var}(XY) \approx Y^2\text{Var}(X) + X^2\text{Var}(Y) + 2XY \cdot \text{Cov}(X, Y)$, and so on. Every implementation we study ships the **independence special case** — the same formulas with the covariance term set to zero — as the operator. The special case is exact iff $\text{Cov}(X, Y) = 0$; applied to correlated operands it understates. Making that hypothesis explicit and checked is the whole paper.

Affine forms / noise symbols. To reason about *which* uncertainty a value carries we use the representation of affine arithmetic (§9.1): a value is an affine form $x = x_0 + \sum_i x_i \varepsilon_i$ over independent unit-variance *noise symbols* ε_i , one per independent measurement source. Then $\text{Var}(x) = \sum_i x_i^2$, $\text{Cov}(x, y) = \sum_i x_i y_i$ (the inner product), and two values are correlated exactly when they share a symbol with nonzero coefficient. The **support** of x is the set of symbols on which it has a nonzero coefficient; **disjoint support** $\Rightarrow$ **zero covariance** (the converse fails — §4.4). The type system tracks an overapproximation of each value’s support (§5).

The Blackwell / data-processing order. An uncertain quantity is an *experiment* (a channel from the true value to

an observation). Blackwell's order compares experiments by informativeness: $B \sqsubseteq A$ iff B is obtainable from A by post-processing through a stochastic channel — a **garbling**, which can only lose information (the data-processing inequality). We call an operation that reports *less* uncertainty than its operands justify an **anti-garbling**; it manufactures information and is forbidden. §4 makes this the soundness criterion, in the variance channel; §9.2 gives its home in quantitative information flow.

4 Anti-garbling as the soundness criterion

§2 showed a family of operations that understate variance on correlated operands. To turn "understate" into a checkable property we need a criterion that says, of an uncertainty operation, whether it is *allowed to produce the answer it produces*. That criterion is not application-specific; it is the information-monotonicity law already standard in quantitative information flow, read in the variance channel.

4.1 Sound operations lose information; anti-garbling creates it

An uncertain quantity is, operationally, an *experiment*: a channel from the unknown true value to an observation. Blackwell's informativeness order compares two such experiments — A is *more informative* than B (written $B \sqsubseteq A$) iff B can be obtained from A by post-processing through a stochastic channel, a **garbling** (Blackwell 1953). Garbling can only discard information; the data-processing inequality is exactly this monotonicity. The quantitative-information-flow community adopts the same order as its refinement order and its soundness backbone: a program transformation is admissible iff it *refines* (is a garbling of) the original — McIver, Morgan, Smith et al. (POST 2014); Alvim et al., *The Science of Quantitative Information Flow* (2020).

We instantiate that discipline for numeric uncertainty propagation:

Soundness criterion (anti-garbling). An uncertainty operation is *sound* iff its output experiment is a garbling of the true joint experiment on its operands — i.e. it never reports *more* information (less uncertainty) than the operands contain. An operation that reports a variance smaller than the truth is an **anti-garbling**; it manufactures information, and no correct program may contain one.

This reframes §2 precisely. `ep_sub(&x, &x)` overstating variance is a *garbling* — it throws information away, which the criterion permits (it is merely conservative). `ep_add(&x, &x)` and `ep_mul(&x, &x)` understating variance are *anti-garblings* — they are forbidden. The add/sub asymmetry of §2.2 is the anti-garbling criterion read directly off the sign of the dropped covariance term.

Scope of this paper. The full Blackwell order is defined on distributions, and the general "reassociation-is-a-garbling" theorem in that setting is developed separately (companion work on non-associative composition). Paper A needs only the **second-moment shadow** of the criterion — the variance channel — because the defect class of §2 lives entirely there: every operation in question propagates *variance*, and the unsoundness is a variance understatement. We therefore state and enforce the criterion on variance, and prove exactly that fragment.

4.2 The scalar operators as an independence claim

Each `ep_*` operation implements the GUM propagation law under an unstated hypothesis. For addition, the general law is $\text{Var}(X+Y) = \text{Var}(X) + \text{Var}(Y) + 2\text{Cov}(X, Y)$; `ep_add` computes $\text{Var}(X) + \text{Var}(Y)$, which equals the truth **iff** $\text{Cov}(X, Y) = 0$. The scalar representation (m, v) cannot express Cov , so the operator cannot condition on it: it asserts independence unconditionally. The defect of §2 is not a wrong formula — each formula is the correct *independent-case* law — but an **unguarded precondition**: the operator is sound on the sub-domain $\text{Cov} = 0$ and is applied on all of it.

Making the criterion checkable therefore reduces to making $\text{Cov} = 0$ a property the type can carry and the compiler can verify. §4.3 pins down exactly what must be verified; §5 carries it in the type.

4.3 The core lemma (kernel-checked)

We model an uncertain value as an **affine form** over independent unit-variance noise symbols $\varepsilon_1, \varepsilon_2, \dots$ — $x = \sum_i c_i \varepsilon_i$ — the representation of affine arithmetic (Comba–Stolfi 1993). The coefficient vector *is* the value's source identity: two values share a source iff they share a nonzero coefficient on the same ε_i . In this model the true variance is $\|c\|^2$, covariance is the inner product $\langle a, b \rangle = \sum_i a_i b_i$, and addition is componentwise (correlation handled by construction). The scalar `ep_add` variance is $\|a\|^2 + \|b\|^2$, which forgets shared coefficients.

Lemma 1 (understatement = twice covariance). For affine forms a, b ,

$$\text{trueAddVar}(a, b) - \text{naiveAddVar}(a, b) = 2\langle a, b \rangle,$$

$$\text{hence } \text{naiveAddVar}(a, b) = \text{trueAddVar}(a, b) \Leftrightarrow \langle a, b \rangle = 0.$$

Proof. $\text{trueAddVar}(a, b) = \|a+b\|^2 = \|a\|^2 + \|b\|^2 + 2\langle a, b \rangle$, and $\text{naiveAddVar}(a, b) = \|a\|^2 + \|b\|^2$; subtract. The general identity is a polynomial identity (ring); the Mathlib-free artifact discharges representative integer witnesses by decide. $\square$

This is checked, not asserted. `SounioAntiGarblingModel.lean` (Lean 4.33.1, `#print axioms = none, sorry = 0`) proves, on witnesses $x = (1, 0), y = (0, 1), z = (2, 1)$:

- `anti_garbling_x_plus_x - naiveAddVar x x < trueAddVar x x (2 < 4)`: the §2 $x+x$ understatement, kernel-checked.

- `anti_garbling_gap_x, anti_garbling_gap_z` — the gap is exactly $2 \cdot \langle \cdot, \cdot \rangle$ at two coefficient scales.
- `sound_under_disjoint` — `naiveAddVar x y = trueAddVar x y`: the scalar add is *exact* when the sources do not overlap.
- `gap_zero_iff_disjoint_witness` — the gap vanishes iff $\langle x, y \rangle = 0$ on these witnesses.

Corollary (the checkable condition). `ep_add` (and, mutatis mutandis, `ep_mul`) is sound on operands a, b iff $\langle a, b \rangle = 0$ — zero covariance. The type system's job is to certify this condition, or reject the operation.

4.4 What the type can actually certify — conservative, not exact

Lemma 1 makes zero *covariance* the exact soundness condition, and covariance is a numeric quantity the type does not know. What the type *can* decide is a **structural** proxy: whether the operands' noise-symbol **supports are disjoint** (share no ε_i). The two are not the same, and honesty about the gap is load-bearing:

Disjoint support $\Rightarrow$ zero covariance, but not conversely. If a and b share no coefficient then $\langle a, b \rangle = \sum_i a_i b_i = 0$. The converse fails: $a = (1, 1)$, $b = (1, -1)$ have overlapping support yet $\langle a, b \rangle = 0$. (This corrects an earlier "sound $\Leftrightarrow$ disjoint" phrasing; the necessary-and-sufficient condition is zero covariance, and disjoint support is sufficient only — codex review, 2026-08-22.)

Consequently the type check is **conservatively sound**: it admits an independence-assuming operator only when it can prove disjoint support, which *implies* $\langle a, b \rangle = 0$, so every admitted operation is genuinely sound (Lemma 1). It may *reject* some sound operations — the coincidentally-orthogonal-but-overlapping case — which §5.5 handles with an explicit escape valve rather than by unsound admission. This is the standard soundness/completeness trade of a static discipline, and we take the sound side: a rejected sound program is a nuisance; an admitted anti-garbling is the bug we exist to prevent.

5 The type system

The criterion of §4 asks for one fact at every independence-assuming operator: are the operand source-supports provably disjoint? We answer it by carrying the source-support in the type and checking disjointness at the operator. The machinery is a set-valued dataflow — the source-identity idea of affine arithmetic (Comba-Stolfi; Goubault-Putot's Fluctuat), but lifted **into the type** and used to **reject** rather than to enclose.

5.1 Types carry a noise-symbol source-set

An epistemic type is $\text{Knowledge}(T, N)$, where N is a **noise-set** drawn from the lattice

$L = \square((S) \cup \{\text{TOP}\}, <:), \quad \{\} <: \text{every finite set } <: \text{TOP, join} = \cup (\text{TOP absorbing}),$

S the set of measurement sources. The implementation (`ns_contract.sio`) represents N as a 3-state handle: $-1 = \top$ (unknown), $0 = \emptyset$ (deterministic / no measured source), >0 an interned nonempty set. The one rule that makes the lattice sound for our purpose:

$\top$ **is never disjoint from anything.** An unknown source-set is treated as potentially sharing every source — the conservative top, not a convenient "assume independent" default (`ns_contract.sio:ns_disjoint, ns_union`).

5.2 Formation

----- (Measure, s fresh)
 $G \vdash \text{measure}(v, \text{sigma}) : \text{Knowledge}(T, \{s\})$
 ----- (Exact)
 $G \vdash \text{certain}(v) : \text{Knowledge}(T, \{\})$

A measurement seeds a fresh singleton source; an exact constant carries the empty set. (`ns_measure` seeds `bit(id)`; the sabotage knob of §8.2 replaces this with $\emptyset$.)

5.3 Transfer

Copy inherits; independence-assuming binary operators union the operand sets, with $\top$ absorbing:

$G \vdash a : \text{Knowledge}(T, N)$
 ----- (Copy / Ident)
 $G \vdash \text{copy}(a) : \text{Knowledge}(T, N)$

Union is the join of L : $\text{ns_union}(a, b) = \text{if } a = \top \sqcap b = \top \text{ then } \top \text{ else } a \cup b$. This is a monotone transfer over the value graph; iterated to a least fixpoint it is the standard Kildall dataflow — realised in `ns_dataflow.sio` as `nsg_propagate`, whose lattice is set-of-sources where the in-tree escape analyzer's is boolean. Same graph, same fixpoint engine, different lattice: the source-identity analysis is the escape analysis with $\cup$ for reachability.

5.4 The checked precondition (the heart)

An independence-assuming operator is well-typed **only** if the operand supports are provably disjoint:

$G \vdash a : \text{Knowledge}(T, N_a) \quad G \vdash b : \text{Knowledge}(T, N_b)$
 $\text{disjoint}(N_a, N_b)$

----- (Add-Indep)
 $G \vdash \text{ep_add}(a, b) : \text{Knowledge}(T, N_a \cup N_b)$

where $\text{disjoint}(N_a, N_b) := N_a \neq \top \sqcap N_b \neq \top \sqcap N_a \cap N_b = \emptyset$. `ep_mul` has the identical side condition (§4's corollary). When the premise $\text{disjoint}(N_a, N_b)$ fails —

overlapping supports, or either operand $\top$ — the operator is rejected:

E230 — anti-garbling: independence-assuming operation over non-disjoint / unknown noise-symbol sets.

`ns_contract.sio:add_flagged` is exactly this predicate (if `ns_disjoint(a,b)` return false else return true), validated by the acceptance controls: `x+x` flagged (shared source), `x+y` accepted (disjoint), `x + unknown` flagged ($\top$ conservative), and `ident(x) + x` still flagged (identity survives a copy). E230 is deliberately a distinct diagnostic from E222 (R-ORIGIN provenance) so the two rules stay causally separable (§8.2, §5.7).

5.5 The escape valve — a proved-disjoint certificate or an explicit covariance

Because the check is conservative (§4.4), some sound programs are rejected. Two admissions recover them without weakening soundness:

1. **A proved-disjoint certificate.** Where the programmer (or an oracle pass) can establish $\langle a, b \rangle = \emptyset$ for overlapping-but-orthogonal operands, a certificate discharges the premise of (Add-Indep) directly — the type admits the operation on the strength of the proof, not the support test.
2. **An explicit correlation-aware operator.** For genuinely correlated operands, the sound path is not to suppress the check but to *take the covariance as an argument*: `gum_s1_add_correlated(a, b,  $\rho$ )` (in-tree, `gum_supplement1.sio`, currently orphaned) propagates $\text{Var}(a) + \text{Var}(b) + 2\rho\sqrt{\text{Var}(a)\text{Var}(b)}$. Its typing rule unions the supports with **no** disjointness premise, because it does not assume independence:

```
G |- a : Knowledge<T, N_a>   G |- b : Knowledge<T, N_b>
----- (
Add-Corr)
G |- add_correlated(a, b, rho) : Knowledge<T, N_a U N_b>
```

The type discipline thus does not forbid correlated arithmetic; it forbids doing it with the operator that *assumes* independence. The programmer's choice of operator becomes a typed claim about correlation, checked against the tracked supports.

5.6 Interprocedural summaries — the load-bearing dependency

The transfer of §5.3 is intraprocedural. Across a call, the source-set of a returned value depends on the callee's body and on which caller-supports flow into which parameters. Two options, and only one is sound: dropping to $\top$ at every call boundary (sound but so conservative it rejects almost everything), or **parametric call-summaries** that substitute caller supports into a callee's abstract source-set. We

take the latter; it is the same summary machinery the compiler's memory-reclamation analysis independently requires (the escape analyzer is intraprocedural today for the same reason), so the engineering cost is shared, not doubled. This is the principal implementation dependency and we flag it as such rather than understate it: without interprocedural summaries the conservative default must be **assume-sharing** — the exact opposite of the library's assume-independence — which is sound but noisy, and the summaries are what make the discipline usable.

5.7 NS and provenance are siblings, not the same rule

The source-set discipline (NS) sits beside the language's provenance discipline (R-ORIGIN, which asks *measured vs derived*), and the two must not be conflated:

Aspect	R-ORIGIN (prove-nance)	NS (noise-symbol)
Question	<i>where</i> did the value come from?	<i>which sources'</i> uncertainty does it carry?
Lattice	scalar origin-kind	source- set (power-set, $\cup$)
Violation	laundering: computed value claims <i>measured</i>	anti-garbling: independence assumed between correlated operands
Diagnostic	E222	E230
Soundness anchor	no-laundering witnesses	SounioAntiGarblingModel (Lemma 1)

They share the dataflow substrate (§5.3) and the `TypeEntry` tag mechanism (a trailing `noise_set_id` field beside provenance `id`), but they are distinct abstract domains with distinct diagnostics, kept causally separable precisely so the evaluation of §8.2 can show that disabling NS removes E230 refusals while leaving E222 refusals intact.

6 Metatheory

We establish that a well-typed program contains no first-order anti-garbling: at every independence-assuming operator, the operands' true covariance is zero, so the propagated variance is exact rather than understated (§4, Lemma 1). The argument has three parts — a mechanized type-safety substrate (§6.2), a sound source-set abstraction (§6.3), and a local soundness criterion already proven (§4.3) — composed in §6.4. Two boundaries are carried as explicit hypotheses of the theorem rather than hidden (§6.5).

6.1 What type safety alone does *not* give

It is worth being precise about the gap the discipline must close, because a conventional type-safety result does **not** close it. Our core calculus already enjoys full type safety (§6.2), and that is not enough: a program can be perfectly

well-typed, never get stuck, preserve its types under reduction, and *still* report an understated variance. Type safety guarantees the metadata stays *valid* (a non-negative variance, a bounded confidence); it says nothing about whether that variance is the *correct* one. Anti-garbling is a soundness property one level above type safety, and it needs the source-set the base calculus does not carry. §6.2 makes this concrete by showing that the mechanized dynamic semantics *is* the defective one.

6.2 The mechanized substrate: type safety, and why it is not soundness

EpistemicEffectsV2.lean formalizes a core epistemic-effects calculus: a Knowledge type `tknow T`, a measure/kraw form carrying scalar GUM metadata `KMeta = {gumVar, conf}`, an effect row with sub-effecting ($\subseteq$), and the arithmetic operators `kadd`, `kmul` typed at `tknow treal (HasTy, :59--89)`. The full type-safety pair is mechanized, Lean 4.33.1:

- **Progress** — `progress' (:223)`, `effect_progress (:301)`: a closed well-typed term is a value or steps.
- **Preservation** — `preservation' (:559)`, `preservation (:626)`: typing and effect rows are preserved under Step, with the usual supporting infrastructure (weakening :420, closed substitution :504, canonical forms :199--223).
- **Metadata validity** — `gAddMeta_valid (:324)`, `gMulMeta_valid (:342)`: the metadata combinators preserve `kvalidm`: `=00m.gumVar000m.conf0m.conf01000`.

Here is the sharp point. The operational combinator the calculus reduces `kadd` through is

```
def gAddMeta (ma mb : KMeta) : KMeta :=
  { gumVar := ma.gumVar + mb.gumVar, conf := if ma.conf <=
    mb.conf then ma.conf else mb.conf }

-- EpistemicEffectsV2.lean:92
def gMulMeta (x : Int) (ma : KMeta) (y : Int) (mb : KMeta) :
  KMeta :=
  { gumVar := y * y * ma.gumVar + x * x * mb.gumVar, ... }
  -- :94
```

`gAddMeta` is `ep_add` and `gMulMeta` is `ep_mul` — **the very operators of §2**, `var_a + var_b` and `y2.var_a + x2.var_b`, with no covariance term. So the mechanized semantics faithfully implements the defect, and `gAddMeta_valid` proves that the defective `add` *preserves validity*: $0 \leq \text{gumVar}$ is maintained even as the variance is understated. This is the formal statement of §6.1 — **validity is preserved; soundness is not** — and it is exactly why a source-set discipline, not a stronger type-safety proof, is required. The substrate is sound *as a type system* and silent *as an uncertainty accountant*.

6.3 The source-set analysis is a sound abstract interpretation

The NS extension of §5 adds a source-set `N` to the Knowledge type and a disjointness premise to `t_kadd/t_kmul`. Its metatheoretic contribution is one abstraction-soundness fact:

Lemma 2 (support over-approximation). For every value, the tracked noise-set `N` over-approximates the value's true noise-symbol support: every source that actually contributes to the value's uncertainty is a member of `N` (with $\top$ the trivially-safe over-approximation).

Argument. The transfer functions of §5.3 are the abstract counterparts of the concrete dependency: `measure` introduces a fresh symbol (the true support of a new measurement); `copy` preserves support; `kadd/kmul` produce a value whose true support is contained in the union of the operands' supports, which the abstract transfer $\cup$ computes exactly, and $\top$ absorbs the unknown case. Each transfer is monotone on the lattice $L = (\square(S) \cup \{\top\}, \sqsubseteq)$ (§5.1), so the analysis has a least fixpoint (Kildall), and monotonicity plus the local containment at each node give the global over-approximation by the standard abstract-interpretation soundness schema. The engine is realized in `ns_dataflow.sio (nsg_propagate, the monotone fixpoint)` with the escape analyzer's proof obligations, `lattice boolean → set`.

Over-approximation is what makes the conservative choices of §5 *sound*, not merely cautious: because `N` can only overstate the true support, $\top$ -is-never-disjoint and the assume-sharing interprocedural default (§5.6) can never mistake a correlated pair for a disjoint one. They can only err toward rejecting a sound program — the completeness side, not the soundness side.

Mechanization. Lemma 2 is now a kernel fact rather than a schema: in `EpistemicEffectsNS.lean` every runtime Knowledge value `kraw v m a` carries its true affine form `a`, its typing rule demands `Covers N a` (every source of `a` is a member of `N`, $\top$ trivially), each transfer preserves it (`covers_single` for `measure`, `covers_union` for `kadd`, `covers_scale+covers_union` for the first-order form of `kmul`), and `support_t_over_approx` reads the over-approximation off any typing derivation — so preservation (§6.4) carries it to every value reached during evaluation.

6.4 The soundness theorem

Theorem (no first-order anti-garbling). Let `e` be a program well-typed under the NS-extended system (§5). Then at every independence-assuming operator (`kadd/kmul`) reached during evaluation of `e`, the operands have zero covariance, and the propagated first-order variance equals the true first-order variance. Equivalently: no reachable `kadd/kmul` in a well-typed `e` is an anti-garbling.

Proof (composition). Take any independence-assuming operator in e . It type-checks, so by (Add-Indep)/(Mul-Indep) its operand source-sets are provably disjoint: $N_a \cap N_b = \emptyset$, both $\neq \top$. By Lemma 2 the true supports are contained in N_a , N_b , hence the true supports are disjoint, hence the true covariance $\langle a, b \rangle = 0$ (disjoint support $\Rightarrow$ zero covariance, §4.4). By Lemma 1 (SounioAntiGarblingModel, kernel-checked) zero covariance makes the scalar $gAddMeta/gMulMeta$ variance *exact* — no understatement. Preservation (EpistemicEffectsNS, $\forall$ preservation) carries the typing, hence the disjointness premise, along each reduction step, so the guarantee holds not just at the source term but at every operator reached during evaluation. $\square$

Mechanization status, stated honestly:

Scope of the mechanization, stated precisely. What is kernel-checked is the *core calculus* of §5 — its type safety, its soundness invariant, and Theorem 6.4 over it. What is *not* kernel-checked is the correspondence between that calculus and the production checker (a manual argument backed by the four controls and the sabotage gate of §8.2), the interprocedural extension (§5.6, not implemented), second-order terms, and the modelling axioms listed below. We do not claim the deployed checker is verified; we claim the rule it implements is.

Every ingredient of the theorem now carries a machine proof (formal/lean4/EpistemicEffectsNS.lean, Lean 4.33.1, Mathlib-free, no sorry; axiom footprint $\subseteq \{\text{propext}, \text{Quot.sound}, \text{Classical.choice}\}$; gate: scripts/ci/ns_metatheory_lean_gate.sh). The calculus makes the ground truth explicit: a runtime Knowledge value carries its true first-order affine form beside the scalar metadata it *reports*, the operational semantics is deliberately the defective one ($gAddMeta = \text{ep_add}$, no covariance term), and soundness is the separate invariant *Exact* — “every value reports its true variance” — which type safety alone does not give (§6.1) and NS typing does. What is **not** mechanized, and stated as such: (i) the correspondence between this core calculus and the production checker’s E230 rule — the wire is source-verified and sabotage-gated (§8.2) but not proven equivalent to HasTy; (ii) interprocedural summaries (§5.6), absent from the calculus; (iii) second-order terms (§6.5); (iv) the noise-symbol axiom itself — distinct measure labels are distinct physical sources, *assumed, not proved*: the type system tracks sources, it does not discover them, and with dishonest labels the calculus under-approximates covariance; (v) the semantics is algebraic — $\langle a, a \rangle$ is the variance under independent unit-variance symbols by definition; distributional adequacy is not proved but **measured** on the RQ4 cohort: a 5 000 $\times$ 1 000-draw Monte Carlo gives $\text{Var_MC}/\text{Var_T} = 0.999$ (0.857–1.158) and the same WARN decision on 99.4 % of patients (paper_A_rq4_mc_adequacy_2026-08-31.md); (vi) *treal* is modelled by *Int* — ring algebra over integer “reals”, exact for first-order propagation, no $\mathbb{R}$ -valued measure theory claimed — and Theorem 6.4’s *Exact* hypothesis is load-bearing (a

Ingredient	Status	
Base calculus progress + preservation	✓ mechanized	—
	EpistemicEffectsV2.lean (Lean 4.33.1)	
$gAddMeta/gMulMeta$ = the §2 operators; validity preserved	✓ mechanized	—
	$gAddMeta_valid$, $gMulMeta_valid$	
Local criterion: disjoint support $\Rightarrow$ zero cov $\Rightarrow$ exact (Lemma 1)	✓ kernel-checked, axiom-free	—
	SounioAntiGarblingModel.lean	
Analysis soundness: N over-approximates true support (Lemma 2)	✓ mechanized	—
	EpistemicEffectsNS.lean: Covers N_a is a typing invariant of runtime values (t_kraw), preserved by every transfer (covers_single, covers_union, covers_scale), extracted by support_over_approx; covers_coeff gives the nonzero-coefficient form	
NS-extended preservation (disjointness premise preserved under Step)	✓ mechanized	—
	EpistemicEffectsNS.preservation (and progress) for the N -annotated $tknow$	
Lemma 1 in general form (all affine forms, not Int witnesses; Mathlib-free)	✓ mechanized	—
	trueVar_append, trueVar_mul (delta method), inner_disjoint	
Exactness preservation: reported variance = true first-order variance along every step	✓ mechanized	—
	exact_preservation: under the premise the defective $gAddMeta/gMulMeta$ are exact	
Theorem 6.4 — no reached independence-assuming operator has correlated operands	✓ mechanized	—
	typed_agfree, soundness_star (along $\Rightarrow *$)	
Partition lemma — a decomposition invariant to its shared sources has $\text{Cov} \leq 0$; the independence-assuming add is then conservative ($x - x \text{ end: Cov} = -\text{Var } a$)	✓ mechanized	—
	partition_iff, inner_nonpos_of_partition, naive_add_conservative_of_partition, inner_eq_neg_var_of_full_partition; with naive_add_understates_iff (anti-garbling $\Leftrightarrow \text{Cov} > 0$)	
Sabotage witness in the kernel: $x+x$ steps to an inexact value and is untypable for every N ; measure $s + \text{measure } s$ and the shared-variable let $x = \text{measure } s$ in $x + x$ untypable at source level; $x + \text{opaque}(y)$ rejected purely by the $\top$ clause (with $x+y$ admitted); $x+y$ stays exact	✓ kernel-checked	—
	$x_plus_x_understates$, $x_plus_x_untypable$, $\text{measure_plus_measure_untypable}$, $\text{let_x_plus_x_untypable}$, $x_plus_top_untypable$, $x_plus_y_exact$	

source-level kraw literal can fabricate metadata). Four adversarial reads 2026-08-30/31 — xAI Grok 4.5, 4.6, 4.6-on-fixes, and Kimi K3 as the independent second vendor: 0 unsound findings, every finding closed by a theorem or stated as a boundary — paper_A_ns_metatheory_xai_review_2026-08-30.md.

6.5 Two boundaries carried as hypotheses

The theorem is deliberately scoped. Both limits are stated as part of the guarantee, not discovered by a referee:

- **Conservative, not complete.** The guarantee is *soundness* (no admitted operator is an anti-garbling), not *completeness* (not every sound operator is admitted). The rule keys on disjoint support, which is sufficient but not necessary for zero covariance (§4.4), so it rejects the overlapping-but-orthogonal case. That case is recovered by the escape valve (§5.5), never by unsound admission. Formally: the theorem quantifies over *admitted* operators; it makes no claim that the admitted set is maximal.
- **The sign of the harm is model-structural (§8.4).** An independence-assuming add understates variance when the shared-source covariance is positive (interval sums, like-signed terms) and over-states it when the covariance is negative (partitions of an invariant, such as the two-compartment phase decomposition). The discipline is indifferent — E230 and exact propagation apply to both — but every “silencing” claim in this paper is a $\text{Cov} > 0$ statement, and we have measured a case where the same defect produces alarm fatigue instead.
- **First-order only.** The soundness criterion (Lemma 1) is exact for the *linear* fragment (kadd, and kmul to first order in the delta method). The nonlinear operators (ep_mul, ep_div, ep_square, ep_sqrt) are delta-method approximations that drop second-order terms, so even under disjoint support a residual second-order discrepancy remains. The theorem therefore guarantees the absence of the *first-order covariance* anti-garbling — the entire defect class of §2 — and explicitly not the truncation error of the delta method, which is a separate, symmetric (non-directional) approximation error and not an anti-garbling. Extending the guarantee to second order is future work (it needs the Hessian/second-moment terms the current metadata does not carry).

7 Implementation

The discipline is implemented in Sounio, a self-hosted language whose checker (Madaros) is itself written in Sounio. The design reuses three things already in the compiler — the TypeEntry tag mechanism, the monotone-dataflow engine, and the provenance rule it sits beside — so the noise-symbol

discipline is an *added tag with a join rule*, not a new type parameter threaded through unification from scratch.

7.1 Where the source-set lives

A Knowledge type’s static data is a TypeEntry record (self-hosted/check/types.sio:139) that already carries a family of index-tags flowing through the compatibility check compat — unit_id(dimensions), refinement_id, algebra_kind, epistemic_meta_id, ontology_id, knowledge_epsilon. The source-set is one more such tag: a trailing field

```
noise_set_id : i64          // -1 = TOP (unknown), 0 = {}, >0
                        = interned nonempty set
```

placed **after** provenance_id (§7.4), defaulting to -1 at every existing TypeEntry construction site (the conservative top). The set itself is interned in a dedicated module self-hosted/check/noise_sets.sio; noise_set_id is a handle into that table, and union/disjoint dereference the handle through the module rather than doing bitwise arithmetic on the id — the id is an identity, not a mask. (The prototype ns_contract.sio uses an inline i64 bitmask, a bounded 64-source stand-in that validates the *semantics*; the compiler uses the interned-handle representation for the *scale*.)

7.2 The join site

The disjointness check has a template already in the tree. compat.sio handles Knowledge compatibility in its TyKnowledge arm (~:230) by comparing knowledge_epsilon, and it handles model-family compatibility in the TyModelFamily arm (:250) with a a.epistemic_meta_id==b.epistemic_meta_id “tags must match or incompatible” test — which is exactly the shape of a source-set join. The independence-assuming binary operators are typed at check.sio (the kadd/kmul join site, ~:18862); the NS rule adds, beside the existing knowledge_epsilon handling, the disjointness premise of §5.4: compute disjoint(noise_set_id(a), noise_set_id(b)) via the noise_sets module, and on failure raise E230 rather than producing a result type.

7.3 Phasing (N1–N4)

The wire is serialized into four behavior-neutral-then-active phases:

- **N1 — representation only.** Add the noise_set_id field and the noise_sets module; default every site to -1. The bootstrap and source build are behaviorally identical to before (no rule consults the field yet). This is the safe, large-surface diff.
- **N2 — transfer.** Seed a fresh symbol at measure; union at kadd/kmul; inherit at copy/ident; the parametric call-summary substitution for interprocedural flow (§5.6).
- **N3 — the gate.** Raise E230 at kadd/kmul when disjointness cannot be proved; the same-source-built sabotage witness (disable only the NS rule → the E230

vanishes while E222 stays — the compiler-level form of §8.2).

- **N4 — regression.** The named CI gate scripts/ci/ns_antigarbling_gate.sh plus the full test suite: compile-fail ns_add_shared_source_rejected.sio (x+x), ns_add_unknown_conservative.sio; run-pass ns_add_disjoint_ok.sio, ns_ident_preserves_source.sio.

Status. N1–N4 are landed in the production checker and verified from a source build (Madaros v0.80.0; the E230 gate at both kadd/kmul sites, the interned noise_sets module, the transfer/join dataflow, and the SOUNIO_NS_DISABLE sabotage knob). The soundness prototypes (noise_symbols.sio, ns_dataflow.sio, ns_contract.sio) remain the kernel-of-the-argument at the analysis level; §8 now reports the *wired-compiler* results alongside them. The remaining [pending wire] items are the genuinely-future ones named in the closing note (interprocedural parameter projection; the two-compartment clinical model landed 2026-08-31, §8.4).

7.4 Coexistence with provenance

noise_set_id is deliberately a *separate* field from the provenance tag provenance_id and from the overloaded knowledge_epsilon (which already multiplexes transport/diagram/fairness/grade confidences — reusing it would collide). The diagnostic E230 is likewise distinct from the provenance diagnostic E222. The two rules share the TypeEntry mechanism and the dataflow substrate but remain independent abstract domains (§5.7), which is what lets the N3 sabotage witness disable one without perturbing the other.

8 Evaluation

We ask four questions:

- **RQ1 — Is the defect real?** Does the anti-garbling class occur in shipping uncertainty code, and how large is the error?
- **RQ2 — Does the type rule cause the rejection?** When the discipline refuses a program, is the refusal attributable to noise-symbol propagation, or could it be an unrelated effect firing coincidentally?
- **RQ3 — How precise is the check?** What sound programs does the conservative rule reject, and does the escape valve recover them?
- **RQ4 — Does it matter?** In a clinical uncertainty model, does the understatement change a decision the system exists to make?

What is measured. The E230 rule is now **wired into the checker at the real kadd/kmul sites and built from source** (Madaros v0.80.0), so RQ1–RQ3 are answered on the wired compiler (below). Every measurement in this section is of the **intraprocedural** checker: the call-summary extension of §5.6 is not implemented, so the false-positive counts of §8.3 are lower bounds and every program crossing a call

boundary is outside the measured scope, backed by the kernel-checked soundness model (SounioAntiGarblingModel.lean) and the analysis prototypes. RQ4 is answered in two halves: the decision-relevant clinical WARN and the exact anti-garbling contraction on a controlled correlated-sum instance are real today, while the end-to-end *two-compartment* patient-flip is now measured on the two-compartment extension (§8.4: 34.2 % silenced in the interval sum, 0 % in the phase decomposition).

8.1 RQ1 — the defect is real, in shipping code

The anti-garbling class is not hypothetical. In the production uncertainty library stdlib/epistemic/knowledge.sio, the same operation x·x has two implementations with different variances — ep_mul(&x,&x) returns 2m²v, ep_square(&x) returns the correct 4m²v (§2.1) — and nothing routes x·x to the sound one. Addition and multiplication understate on any correlated operands; the error is exactly the dropped covariance term 2·Cov (§2.2, Lemma 1). For maximally correlated operands the understatement is a factor of two in variance — a factor of $\sqrt{2}$ in the reported standard deviation — in the *optimistic* direction. The library's own test suite (knowledge.sio:290–310) exercises these operators only on independent operands, so the defect ships untested (§2.3).

This establishes the target: a real, silent, safety-relevant unsoundness that current testing does not surface and that no type in the library distinguishes from correct code.

8.2 RQ2 — the rejection is caused by noise-symbol propagation

A type rule that rejects x+x is only meaningful if the rejection is *because of* the shared source, not an artifact of some other check firing. We establish causality with a **sabotage control**: a single knob that disables noise-symbol set-propagation (measurement nodes seed $\emptyset$ instead of a fresh symbol) while leaving every other rule intact. If the x+x refusal is caused by NS, flipping the knob must make exactly that refusal vanish and leave unrelated refusals standing.

ns_contract.sio encodes this as five acceptance controls. Run today, verbatim:

```
$ ./bin/souc run docs/research/sounio/ns_contract.sio
NS contract -- five acceptance controls
1 x+x flagged (shared source): PASS
2 x+y accepted (disjoint cert): PASS
3 unknown conservative (flagged): PASS
4 ident(x)+x flagged (identity survives): PASS
5 sabotage: x+x NOT flagged (refusal vanishes): PASS
ALL FIVE CONTROLS PASS
```

Reading the controls against the type rules of §5:

Control 5 is the load-bearing one: with set-propagation removed, x+x is no longer flagged, so the refusal in control 1 is *attributable to* the propagated source-set and not to a

Control	Tests	§5 rule exercised
1 $x+x$ flagged	shared source $\Rightarrow$ E230	(Add-Indep) premise fails
2 $x+y$ accepted	disjoint supports $\Rightarrow$ admitted	(Add-Indep) premise holds
3 $x+\top$ flagged	unknown never disjoint	§5.1 $\top$ -conservatism
4 $\text{ident}(x)+x$ flagged	identity survives a copy	§5.3 (Copy) transfer
5 sabotage $\Rightarrow x+x$ clean	refusal is caused by NS	causality witness

coincident effect. The independent dataflow prototype confirms the same distinction on a value graph rather than on scalar handles:

```
$ ./bin/souc run docs/research/sounio/ns_dataflow.sio
NS dataflow analysis (source-set fixpoint over the value
graph)
s1 = ADD(x, x): FLAGGED anti-garbling (inputs share a source
)
s2 = ADD(x, y): clean (disjoint sources)

s1 = x+x (shared source) is flagged; s2 = x+y (disjoint)
is clean — the same verdict, reached by a monotone least-
fixpoint over the graph (§5.3), which is the compile-time
form of the check.
```

On the wired compiler, the same causality holds — measured. Against the source-built checker (Madaros v0.80.0), $x + x$ (shared source) and $x + u$ (u an unknown-support call return) both raise error[E230]; under SOUNIO_NS_DISABLE=1 — which disables *only* the anti-garbling refusal on an otherwise-identical build — both E230s vanish, while an R-ORIGIN fixture (`r_origin_measured_on_sum.sio`) on the *same* build still raises E222. This is the compiler-level form of the causality claim: the refusal is attributable to noise-symbol propagation and is causally separable from the provenance rule. It runs as `scripts/ci/ns_antigarbling_gate.sh` (all controls pass). The prototype witness above and this wired witness agree.

8.3 RQ3 — precision: what the conservative rule costs

The check keys on disjoint *support*, which is sufficient but not necessary for zero covariance (§4.4). It is therefore sound but incomplete: it rejects the overlapping-but-orthogonal case — operands sharing a symbol whose signed coefficients cancel, e.g. $a = x_1+x_2$, $b = x_1-x_2$, with $\langle a, b \rangle = 0$.

Measured on the wired compiler. Across the 95 Knowledge arithmetic run-pass tests in the suite, the gate raises E230 on 6, passes 77, and leaves 11 **pre-existing failures unchanged** (all 11 persist under SOUNIO_NS_DISABLE, so none is attributable to the rule). Every one of the 6 vanishes under the disable knob (all NS-caused). They break down as:

5 dataflow-witness tests that deliberately exercise a now-refused op (a shared $s+s$ self-add or an unknown-support operand) — reconciled by running them under SOUNIO_NS_DISABLE=1 (`scripts/ci/ns_dataflow_trace_gate.sh`); and 1 clinical propagation model refused for the *interprocedural-parameter* reason (§10), not a construction case. Two false positives that first surfaced — a struct-literal `Knowledge{..}` and a module-level `let Knowledge`, both defaulting their source-set to $\top$ — were genuine seeding gaps and were fixed (those paths now seed $\emptyset$), after which their programs type-check. The residual non-disjoint false-positive class (overlapping-but-orthogonal, and cross-parameter correlation the callee cannot see) is the interprocedural gap, out of this slice.

The escape valve (§5.5) bounds the cost: a rejected sound program is recovered either by a proved-disjoint certificate (discharging the premise on the strength of $\langle a, b \rangle = 0$) or by switching to the correlation-aware operator `add_correlated(a, b, p)`, which carries the covariance explicitly and needs no disjointness premise. So the conservative rule never *blocks* correct code; it forces correlated arithmetic to be written with the operator that does not assume independence. The false-positive rate on real uncertainty code is the 6-of-95 above (and every one is a *characterized* refusal — reconcilable witness or the known interprocedural gap — not an unexplained rejection); the finer breakdown of *which* fix (disjointness certificate vs. correlation-aware operator) each site takes is a study for a larger corpus.

8.4 RQ4 — it changes a clinical decision

The stakes are set by a real, running model. `examples/vancomycin_auc_epistemic.sio` (a run-pass example; it runs under the `lean_single` engine — the self-hosted Madaros compiler fails closed on builtin `Knowledge<T>` arithmetic, E245/E004, #1706 — and its engine-portable twin `examples/vancomycin_auc_affine.sio` reproduces the same receipt on both engines through `stdlib/epistemic/affine`) propagates GUM uncertainty through a vancomycin AUC-guided therapeutic-drug-monitoring chain for a discriminating patient (65 yr male, 70 ± 1 kg, SCr 1.40 ± 0.14 mg/dL, 500 mg q12h):

```
CrCl (Cockcroft-Gault) = 52.1 mL/min, u(CrCl) = 5.2
CL (Matzke 1984) = 2.22 L/h, u(CL) = 0.22
AUC_0_2 (q12h) = 450 mg*h/L, u(AUC) = 44 =>
95% CI [362, 538]
```

The point estimate $AUC = 450$ reads **therapeutic**; the credible interval [362, 538] **crosses the 400 subtherapeutic boundary**, and the epistemic model raises WARN: possible subtherapeutic. The entire clinical value of propagating uncertainty is that this WARN fires where the point estimate is silent — the decision-flip the deployed point-estimate systems (InsightRx, DoseMeRx, JPKD) cannot produce.

The anti-garbling threat to this WARN. The width of the credible interval *is* the propagated uncertainty. Any operation that understates variance shrinks the interval toward

the point estimate and can pull its lower bound back across 400 — silencing the WARN. The bite lands wherever the model combines two quantities that share a measured source. On a controlled instance: summing two AUC contributions that both descend from the same measured clearance, $\text{add}(\text{auc}_a, \text{auc}_b)$ with $\text{Cov}(\text{auc}_a, \text{auc}_b) > 0$, the independence-assuming ep_add omits 2-Cov; by Lemma 1 the reported variance is understated by exactly that term, and the interval half-width contracts by $\sqrt{1-2\text{Cov}/\text{Var_true}}$. For strongly correlated compartments ($\rho \rightarrow 1$) this is the factor-of- $\sqrt{2}$ SD contraction of §8.1 — enough to move a lower bound of 362 above 400 and convert a WARN into a false THERAPEUTIC. When $\text{Cov} < 0$ the same omission errs the other way — the interval *widens* and WARNs become spurious — so the contraction is a $\text{Cov} > 0$ statement; §8.4's measurement below exhibits both signs.

Measured (2026-08-31). The two-compartment extension now exists and the flip rate is measured — docs/research/sounio/rq4_vanco_two_compartment_flip.sio, one deterministic cohort of 5,000 patients (weight 45–120 kg, SCr 0.6–2.6 mg/dL, Q and Vp $\pm 30\%$ about population, $u(\text{weight}) = 1$ kg, $u(\text{SCr}) = 10\%$, $u(Q) = u(Vp) = 20\%$; 500 mg q12h; 909 true WARNs among 1,669 therapeutic-window point estimates), propagated three ways: first-order affine forms over the measured sources (the truth, **T**), the shipped scalar ep_ chain (**N**), and exact operands with an independence-assuming *final add only* (**S**, isolating Lemma 1's 2-Cov). Two shared-source sums a PK library actually performs:

shared-source sum	true WARN	silenced by the naive add	Var naive/true	ratio
B — inter-val sum AUC(0--12) + AUC(12--24), same CL ($\rho = 1$)	909	311 = 34.2 %	0.500	
A — two-compartment phase sum $A/\square + B/\square$	909	0 (62 spurious instead)	1.204 (final add); 300.7 (whole chain: 1,894 spurious WARNs, 38 % of the cohort)	

B is the anti-garbling this section feared, at the size it feared: with $\rho = 1$ and equal terms Lemma 1 gives exactly half the variance (the $\sqrt{2}$ contraction of §8.1), and it silences one true WARN in three. **A is an honest null in the feared direction** — and a finding: the phase covariance is *negative* in 5,000/5,000 patients, because AUC is invariant to Q and Vp and the decomposition into phases is a partition of that invariant — whatever Q and Vp move into one phase they move out of the other. This is now a theorem of the calculus, not an observation: *inner_nonpos_of_partition* — if every shared source of a sum is a *partition source* (the sum is

invariant to it: $\text{coeff}(a \text{ ++ } b) s = 0$, equivalently $\text{coeff} b s = -\text{coeff } a s$, partition_iff), then $\langle a, b \rangle \leq 0$ and the independence-assuming add is conservative ($\text{naive_add_conservative}$ at the fully-partitioned end $\langle a, b \rangle = -\text{Var } a$). The sign of the harm is predictable from the model's invariances alone, before any coefficient is computed. There the independence-assuming add *over-states* variance, and across the whole chain the over-statement compounds to 300×: garbling rather than anti-garbling, and a different clinical harm (alarm fatigue: 1,894 spurious WARNs) from the same defect. The sign of the covariance decides which harm you get; the discipline does not need to know the sign — E230 rejects the shared-source add either way, and exact propagation ($\text{exact_preservative}$ is right in both directions. The first-order truth is the sampling truth here: a Monte Carlo of the same cohort (1 000 draws per patient) gives $\text{Var_MC}/\text{Var_T} = 0.999$ (0.857–1.158), the same WARN decision on 99.4 % of patients, and $\text{Var_N}/\text{Var_MC} = 300.9$ — the 300× is not a linearisation artefact (paper_A_rq4_mc_adequacy_2026-08-31.md). Full record and reproduce line: paper_A_rq4_two_compartment_flip_2026-08-31.md.

8.5 Threats to validity

- **Construct.** The soundness criterion is enforced on the *variance* (second-moment) channel (§4.1). Non-Gaussian or heavy-tailed uncertainty is under-described by variance; the criterion catches variance understatement, not every distributional anti-garbling.
- **Internal.** RQ2's causality rests on a single sabotage knob; it is now demonstrated at both the analysis level (the prototype) and the **compiler level** (E230 vanishes, E222 remains, same source build) — the two agree. A knob is still one mechanism; a second, independent construction of the causality claim would strengthen it.
- **External.** RQ1 quantifies one library; the *class* (independence assumed and unchecked) is general to GUM-style propagation, but we measure one instance. RQ4's magnitude is exact on a controlled instance, not a patient-cohort flip rate.
- **Scope of the guarantee.** Soundness holds on the linear fragment; nonlinear operators (mul , div , square , sqrt) retain a delta-method second-order residual even under disjoint support (§6.3) — the type prevents the *first-order* covariance anti-garbling, not the truncation error of the delta method itself.

9 Related work

Our contribution sits at the intersection of three lines, none of which occupies it. We state each neighbor and the precise delta, so the claim is neither the tracking (30 years old) nor the soundness frame (standard in another domain) but their

combination as a compile-time type rule in an uncertainty-typed language.

9.1 Affine arithmetic and zonotopic static analysis

Noise-symbol identity is the defining device of **affine arithmetic** (Comba & Stolfi 1993): a quantity is $x_0 + \sum_i x_i \varepsilon_i$, and shared ε_i between two quantities are exactly their correlation, so $x - x = 0$ and correlated errors do not inflate independently. **Goubault & Putot's** zonotopic abstract domains and the **Fluctuat** analyzer (*Static Analysis of Finite Precision Computations*, VMCAI 2011; *Perturbed affine arithmetic for invariant computation*, 2008; the logical-product zonotope intersection, 2010) build a static analysis on precisely this representation, tracking correlations between program variables through shared noise symbols to bound finite-precision error in C/Ada.

This is the closest prior work on *source identity*, and we reuse its core idea. The delta is threefold: (i) Fluctuat's noise symbols live in an **external analyzer** producing an enclosure; ours live **in the type** and are part of the program's interface. (ii) Fluctuat **reports** a bound; we **reject** — an independence-**assumption** over correlated operands is a *type error* (E230), not a wider interval. (iii) Fluctuat targets roundoff of a fixed computation; we target the *soundness of uncertainty propagation itself*, where the failure is a library computing the wrong variance formula, and the fix is a checked precondition plus a correlation-aware operator. "We track source identity" is not our claim; "we make it a typed, rejecting precondition in an uncertainty language" is.

9.2 Quantitative information flow and the Blackwell order

The soundness criterion — an operation may lose information but never create it — is the **data-processing / Blackwell informativeness order** (Blackwell 1953), and the **quantitative information flow** community has made it the backbone of its refinement theory: McIver, Morgan, Smith, Espinoza & Meinicke, *Abstract channels and their robust information-leakage ordering* (POST 2014), and Alvim, Chatzikokolakis, McIver, Morgan, Palamidessi & Smith, *The Science of Quantitative Information Flow* (Springer 2020), identify program refinement with channel garbling and use post-processing monotonicity as the soundness condition — A refines B iff B is a garbling of A.

We adopt this frame wholesale and say so. The delta is the *domain and the mechanism*: QIF orders **confidentiality channels** and measures **leakage**; we order **uncertainty propagation** in the **variance channel** and enforce the anti-garbling prohibition as a **static type rule** at arithmetic operators. Reading `ep_add`'s variance understatement as an anti-garbling is, to our knowledge, a new instantiation of the QIF/Blackwell discipline; it is not a new order. (The lift

of this frame to non-associative composition — where re-association itself becomes a garbling governed by the octonion associator — is a separate contribution and not claimed here.)

9.3 Uncertainty-typed languages and libraries

The systems that carry uncertainty in the type or value are our *host* setting, and are precisely the ones that do **not** track source identity. **Uncertain(T)** (Bornholt, Mytkowicz & McKinley, ASPLOS 2014) represents a value as a sampled distribution and computes over it, but a Monte-Carlo product of a variable with itself does not know it is the same variable unless the samples are shared by construction, and nothing types the fusion. **Measurements.jl** (Giordano) propagates GUM uncertainty and *does* track correlations at runtime via a derivative graph — but as a numeric result, not a type, and with no compile-time rejection of an independence-assuming path. GUM implementations (ISO/IEC 98-3) and Ferson **p-boxes** give the underlying arithmetic; they are libraries, and fusing two quantities wrong is a numeric mistake, never a type error. Our contribution is exactly the missing enforcement: the source-set in the type, the operator's independence assumption as a checked precondition.

9.4 Type systems for numerical error

A parallel line puts *numerical* properties in the type. **Numerical Fuzz** (Numerical Fuzz: a type system for rounding-error analysis, 2024) and **Bean** (a language for backward error analysis, 2025), and the broader *type-based approaches to rounding-error analysis* (2025), use linear/sensitivity typing to bound roundoff. These share our shape — a numeric soundness invariant carried by the type — but a different invariant: roundoff magnitude, not correlation-soundness of uncertainty propagation. The two are complementary; a value could in principle carry both a sensitivity bound and a noise-set. We borrow the discipline (numeric-property-in-the-type) and contribute a distinct invariant and its dataflow.

9.5 Information-flow types and probabilistic programming

The type machinery closest to ours is **information-flow / taint typing**: a set-valued lattice propagated through the program, with a check at sinks. The noise-set is an IFC-style lattice whose "labels" are measurement sources and whose "sink" is an independence-assuming operator; the reading is novel (covariance-soundness rather than confidentiality), the machinery is familiar, and we position NS as an IFC-shaped discipline for a numeric soundness property. Finally, **probabilistic programming** (Stan, Pyro) *can* express correlated uncertainty via hierarchical models, but at inference time: a posterior fuses correlations correctly only when the model is hand-structured to, and fusing wrong is a modelling choice, never a compile-time error. We target the opposite regime — cheap, first-order, static, and *rejecting* —

where the guarantee is that the unsound path does not type-check.

10 Limitations

We collect the boundaries stated locally through the paper, so the guarantee's shape is in one place.

- **Conservative, not complete (§4.4, §6.5).** The check keys on disjoint *support*, which is sufficient but not necessary for zero covariance. It therefore rejects the overlapping-but-orthogonal case ($a = x_1 + x_2, b = x_1 - x_2, \langle a, b \rangle = 0$). Such programs are recovered by the escape valve (a proved-disjoint certificate or the correlation-aware operator, §5.5), never by unsound admission. The guarantee is soundness, not maximality of the admitted set.
- **First-order / variance channel only (§6.5, §8.5).** Soundness is exact for the linear fragment; nonlinear operators (`mul`, `div`, `square`, `sqrt`) are delta-method approximations that drop second-order terms, so a residual second-order discrepancy survives even under disjoint support. It is a symmetric approximation error, not a directional anti-garbling, and the type prevents the *first-order covariance* anti-garbling — the entire defect class of §2 — not the truncation error itself. Non-Gaussian or heavy-tailed uncertainty is likewise under-described by variance; the criterion is a second-moment one.
- **Interprocedural summaries are the load-bearing dependency (§5.6, §7.3).** The transfer is intraprocedural; sound cross-call source tracking needs parametric call-summaries. Without them the sound default is *assume-sharing* (drop to $\top$ at call boundaries), which is sound but noisy. Building the summaries — shared with the compiler's memory-reclamation analysis — is the principal engineering cost and is part of the pending wire.
- **Unknown correlation beyond $\{0, 1\}$ (§9.5).** The escape valve's correlation-aware operator takes a known ρ . When the correlation is *unknown* (not zero, not one, not a given value), sound propagation needs Fréchet bounds, and the correlation assumption should itself become a tag on the type. We do not model this; it is a stated gap.
- **Which shared-source sums real PK models perform is not measured (§8.4).** The flip rate is reported for two sums — an interval sum ($\rho = 1$, understates) and a phase partition ($\text{Cov} < 0$, over-states) — on one synthetic cohort (deterministic LCG, seed 20260831, released with the artifact). How often deployed PBPK code performs the former versus the latter, and how the rates change once the correlation-aware operators the discipline encourages are adopted, is future

work; the paper's claim is the mechanism and its two signs, not a population prevalence.

- **Evaluation is one library, one wired compiler, and a 95-test suite (§8).** RQ1 quantifies one shipping library; the class is general to GUM-style propagation but measured on one instance. RQ2's causality is established at the analysis level (the sabotage control) and awaits its compiler-level form. The corpus false-positive rate (RQ3) and the full two-compartment clinical flip rate (RQ4) are measured on the wired compiler and on the two-compartment extension respectively (§8.4); the RQ4 cohort is synthetic (a deterministic LCG grid over plausible ranges), not a patient registry.
- **Confidence decay is heuristic.** The confidence field's per-operation decay ($\times 99/100, \times 98/100$) is not derived from a principle; it is orthogonal to the variance soundness this paper establishes, but it is drift until derived, and we do not defend it.

11 Conclusion

Uncertainty-typed languages promise to carry $\pm \sigma$ in the type and propagate it automatically. They keep the promise only where their unstated hypothesis holds — that the operands of every operation are independent — and they break it silently everywhere else, because the failure mode is a *tighter* error bar, the one answer that never looks wrong. We showed the failure is not hypothetical: in a shipping library the same operation $x \cdot x$ has two variances, and nothing routes the program to the sound one.

The fix is to name the hypothesis and check it. Reading uncertainty propagation through the Blackwell / data-processing order already standard in quantitative information flow, understating variance is an *anti-garbling* — manufacturing information — and no correct program may contain one. We carry the noise-symbol source-set of each value in its type, reusing affine arithmetic's source identity but lifting it from an external analyzer into the type, and make the independence assumption of arithmetic a *checked precondition*: an independence-assuming operator over operands whose sources are not provably disjoint is a type error, discharged only by a proof of disjointness or by switching to an operator that takes the correlation explicitly. The core soundness criterion is kernel-checked, and the discipline is wired into the production checker and verified from a source build: on the compiler itself, the number that is too precise to be true no longer type-checks.

The guarantee is deliberately narrow and honestly bounded — first-order, conservative, variance-channel — and it eliminates exactly the defect class it targets: the number that looks too precise to be true is the one the compiler now refuses to print. Two directions extend it. Downward, to second order: carrying the delta-method's dropped terms turns

the first-order guarantee into a full one. Outward, to non-associative composition: when the affine coefficients live in a non-associative algebra, *reassociating* a product becomes a

garbling in its own right, governed by the octonion associator — the point where this discipline meets a genuinely open question about the Blackwell order and algebraic curvature, and the subject of separate work.